

Mini-CLIP

Projektdokumentation: Re-Implementierung der Kernidee von CLIP
(Radford et al., 2021) auf kleinerem Maßstab

Daniel Alexander Panczyk
Technische Hochschule Mittelhessen
Modul: Konzepte des Deep Learning (KDDL)

Sommersemester 2026

Zusammenfassung

In diesem Projekt haben wir die Trainingsmechanik von CLIP (*Contrastive Language-Image Pre-training*) komplett selbst nachgebaut und so weit verkleinert, dass sie sich auf einer normalen CPU trainieren lässt: zwei Encoder mit rund einer Million Parametern, symmetrischer InfoNCE-Loss, gelernte Temperatur und Zero-Shot-Klassifikation über Text-Prompts. Um Implementierungsfehler von Skalenproblemen zu trennen, haben wir zweistufig validiert. Auf einem selbst erzeugten synthetischen Datensatz erreicht unser Modell 100 % Zero-Shot-Accuracy, was die Korrektheit der Implementierung belegt; auf Flickr8k lernt es Retrieval klar über dem Zufallsniveau, overfittet aber deutlich. Der Vergleich mit dem vortrainierten Original auf demselben Test-Split (R@1: 0,9 % vs. 40 %) und ein eigenes Experiment zur Batchgröße bestätigen die Kernaussage des Papers: Die Leistung kommt vor allem aus der Skalierung von Daten, Modellgröße und Rechenleistung und weniger aus der Architektur.

1 Einleitung und Zielsetzung

CLIP [1] lernt Bildverständnis nicht aus gelabelten Klassen, sondern aus 400 Millionen Bild-Text-Paaren aus dem Internet – und kann danach neue Aufgaben per Zero-Shot über Text-Prompts lösen. Das Paper komplett zu reproduzieren war für uns von vornherein unrealistisch, weil schlichtweg die Rechenleistung fehlt: Allein das größte Modell trainierte 18 Tage auf 592 V100-GPUs. Daraus ergab sich unsere Projektfrage:

Lässt sich die Mechanik von CLIP – zwei Encoder, gemeinsamer Embedding-Raum, kontrastiver Loss, Zero-Shot-Klassifikation – so weit verkleinern, dass sie auf einer gewöhnlichen CPU vollständig nachvollziehbar wird, und lassen sich die Kernaussagen des Papers auch in diesem Maßstab empirisch belegen?

Herausgekommen ist **Mini-CLIP**¹: eine Eigenimplementierung in PyTorch mit rund einer Million Parametern – die Originale liegen je nach Variante bei etwa 102 bis 428 Millionen. (Unsere Parameterzahl hängt vom Vokabular ab: 0,76 Mio. auf dem synthetischen Datensatz, 1,33 Mio. auf Flickr8k.) Der Wort-Tokenizer sowie Teile der Daten-Hilfsklassen wurden KI-gestützt umgesetzt (Details in Kap. 7). Wir validieren das Modell auf einem synthetischen Datensatz und auf Flickr8k, vergleichen es mit dem vortrainierten Original und untersuchen zusätzlich, wie sich die Batchgröße in unserem kleinen Maßstab auswirkt.

2 Grundlagen (Zusammenfassung des Papers)

CLIP (*Contrastive Language-Image Pre-training*) besteht aus einem Bild-Encoder (ResNet- oder ViT-Varianten) und einem Text-Encoder (12-Layer-Transformer, BPE-Tokenizer), die beide per

¹Der Name lehnt sich direkt an den Modellnamen des Papers an – ein CLIP im Kleinformaat.

linearer Projektion in einen gemeinsamen Embedding-Raum abbilden (Abb. 1). Trainiert wird kontrastiv: Von den $N \times N$ möglichen Bild-Text-Kombinationen eines Batches sollen die N richtigen Paare (die Diagonale der Ähnlichkeitsmatrix) eine hohe Kosinus-Ähnlichkeit bekommen, alle anderen eine niedrige. Die übrigen Texte im Batch dienen dabei automatisch als Negativbeispiele (*In-Batch-Negatives*). Der Loss ist eine symmetrische Kreuzentropie (InfoNCE) [2]; die Temperatur wird als log-parametrisierter Skalar mitgelernt (Start 0,07, Begrenzung auf 100). Interessant fanden wir, dass die Autoren zuerst einen generativen Ansatz probiert hatten, bei dem die Caption exakt vorhergesagt werden sollte – der skalierte aber schlecht, und das kontrastive Ziel war laut Paper rund viermal dateneffizienter. Für die Zero-Shot-Klassifikation setzt man die

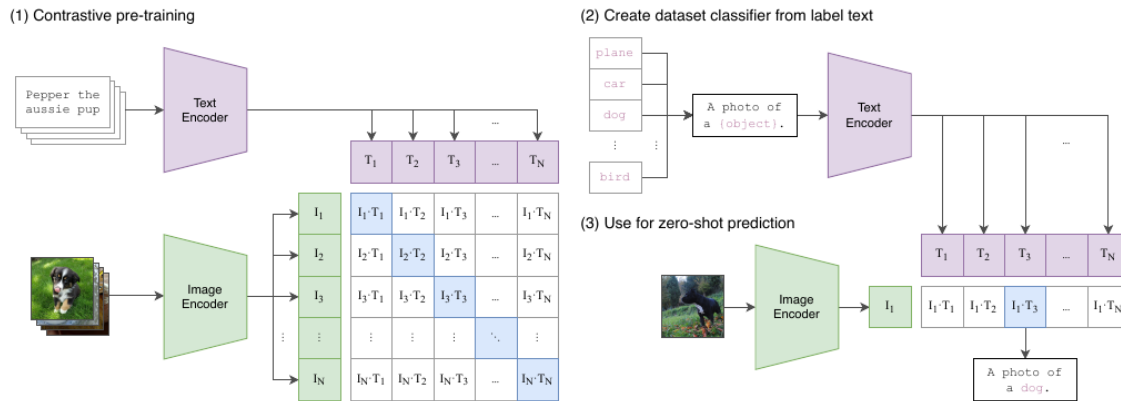


Abbildung 1: Der CLIP-Ansatz: kontrastives Pre-Training (links) und Zero-Shot-Klassifikation über Text-Prompts (rechts). Abbildung aus [1], Figure 1.

Klassennamen in Prompts ein („a photo of a {label}“), schickt sie durch den Text-Encoder und vergleicht die Embeddings mit dem Bild-Embedding. Damit erreicht das beste Modell 76,2% Top-1 auf ImageNet, also das Niveau des überwachten ResNet-50, ohne dessen Trainingsdaten gesehen zu haben; außerdem ist es ungewöhnlich robust gegenüber Verteilungsverschiebungen. Für unser Projekt waren drei Bausteine direkt übernehmbar (Loss, gelernte Temperatur, Zero-Shot-Verfahren) – und eine Limitation entscheidend: der enorme Trainingsaufwand. Er ist der Grund für unsere Reduktion auf ein Mini-CLIP.

3 Konzeption und Design-Entscheidungen

Als Erstes haben wir die Inhalte des Papers in zwei Kategorien sortiert. Zur **maßstabsunabhängigen Mechanik** zählen für uns der symmetrische InfoNCE, die L2-Normalisierung, die gelernte Temperatur mit Paper-Initialisierung, die lineare Projektion, das Zero-Shot-Verfahren über Prompts und das Trainingsverfahren (AdamW + Warmup + Cosine-Decay). Diese Teile haben wir exakt übernommen. Zu den **Skalierungsentscheidungen** gehören Encoder, Tokenizer, Auflösung, Batchgröße und Datenmenge – diese Teile haben wir ersetzt. Unsere Überlegung dahinter: Wenn die Mechanik richtig implementiert ist, muss sie auch im Kleinen messbar funktionieren. Der Abstand zum Original zeigt dann, wie viel die Skalierung ausmacht. Tabelle 1 fasst die Entscheidungen zusammen.

Begründungen. Für den Bild-Encoder haben wir uns für ein einfaches CNN entschieden statt ResNet oder ViT. ViTs brauchen sehr viele Daten – bei uns ist die Lage genau umgekehrt. Vier Stride-2-Blöcke reduzieren die 64 px auf 4×4 Feature-Maps. Den Text-Transformer haben wir als Architekturklasse beibehalten, allerdings mit zwei Abweichungen: Mean-Pooling statt [EOS]-Token (bei kurzen Captions stabiler) und bidirektionale statt maskierter Attention, weil

Tabelle 1: Architekturvergleich und Verkleinerungsentscheidungen.

Komponente	Original-CLIP	Mini-CLIP (unsere Wahl)
Bild-Encoder	ResNet-50 ... RN50x64 / ViT	CNN, 4 Conv-Blöcke (32→64→128→256)
Text-Encoder	Transformer, 12 Layer, 63M	Transformer, 2 Layer, $d = 128$, 4 Heads
Tokenizer	BPE (49 152 Vokabeln)	Wort-Tokenizer aus Trainings-Captions
Text-Pooling	[EOS]-Token-Aktivierung	Mean-Pooling über Nicht-Padding-Tokens
Auflösung / Embedding	224–336 px / 512–768	64 px / 256
Parameter	≈102–428 Mio. (RN50 ... ViT-L/14)	0,76 Mio. (Synthetik) / 1,33 Mio. (Flickr8k)
Trainingsdaten	~400 Mio. Paare	1 500 synthetisch / 6 000 Flickr8k

die Maskierung im Original nur für die optionale LM-Initialisierung gebraucht wurde. Ein Wort-Tokenizer statt BPE reicht bei ein paar tausend verschiedenen Wörtern aus. Alle Hyperparameter liegen zentral in einer Config-Dataclass mit festem Seed. Die Lernrate $5 \cdot 10^{-4}$ haben wir über vier Größenordnungen ausgetestet (Notebook, Abschn. 6b), den Temperatur-Startwert $1/0,07$ mit einem Kontrollexperiment gegen einen neutralen Start überprüft (Notebook, Abschn. 5). Dass der Loss überhaupt Negative braucht – eine *Positive-only*-Variante kollabiert ohne Trenn-Margin – zeigen wir im Notebook (Abschn. 2) an einem Minimalbeispiel.

3.1 Zweistufige Datenstrategie

Stufe 1 – SyntheticShapes (Korrektheitsbeweis): ein selbst konstruierter Datensatz aus programmatisch gerenderten Formen (Kreis, Quadrat, Dreieck, Stern) in fünf Farben, mit Captions im Format „a photo of a {color} {shape}“ (Abb. 2). Er läuft in Sekunden, hat saubere Klassen und dient gleichzeitig als Kontrolle: Wenn das Modell hier nicht fast perfekt lernen würde, wüssten wir, dass der Fehler in der Implementierung liegt und nicht an den Daten. **Stufe 2 – Flickr8k (Realitätstest):** 8 000 reale Fotos mit je fünf menschlichen Beschreibungen [4]. Gesplittet wird auf Bildebene, damit keine Captions desselben Bildes in Training und Test landen (Data Leakage); im Training nutzen wir alle fünf Captions als sprachliche Augmentierung, evaluiert wird strikt 1:1.



Abbildung 2: Beispiele des selbst konstruierten synthetischen Datensatzes (SyntheticShapes).

4 Implementierung

4.1 Projektstruktur

Wir trennen wiederverwendbare Module (`src/`) von ausführbaren Skripten (Tab. 2). Das Notebook nutzt dieselben Module wie die Trainingsskripte, Änderungen am Code wirken sich also auf beide aus.

4.2 Modell und Loss

Beide Encoder enden in einer linearen Projektion auf 256 Dimensionen; die Ausgaben werden L2-normalisiert, sodass Skalarprodukte Kosinus-Ähnlichkeiten entsprechen. Die Temperatur haben wir wie im Original-Code als Log-Parameter angelegt (`logit_scale = log(1/0,07)`, vor der

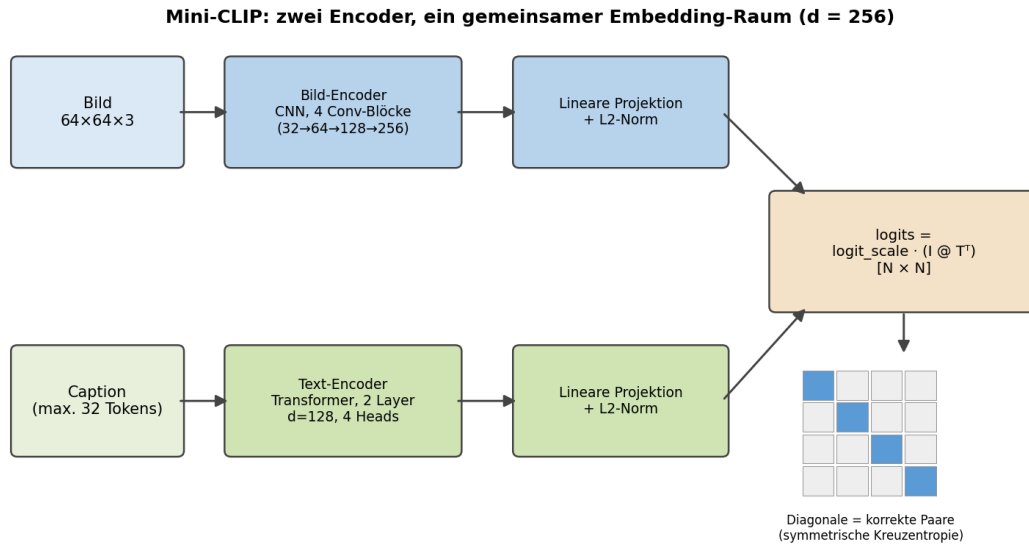


Abbildung 3: Architektur des Mini-CLIP (eigene Darstellung): zwei Encoder, lineare Projektion in einen gemeinsamen 256-dimensionalen Raum, kontrastives Training auf der $N \times N$ -Ähnlichkeitsmatrix.

Tabelle 2: Projektstruktur und Verantwortlichkeiten.

Datei	Zweck
<code>config.py</code>	alle Hyperparameter als Dataclass (Tab. 3), fester Seed
<code>src/model.py</code>	Bild-Encoder, Text-Encoder, MiniCLIP, <code>clip_loss</code>
<code>src/tokenizer.py</code>	Wort-Tokenizer (Vokabular-Aufbau, Encoding, Persistenz)
<code>src/data.py</code>	<code>SyntheticShapes</code> , Flickr-Loader, <code>Collate</code>
<code>src/evaluate.py</code>	Retrieval Recall@K, Zero-Shot-Klassifikation
<code>prep_parquet.py</code>	Flickr8k-Rohdaten $\rightarrow$ npy/json-Cache
<code>train.py</code> , <code>train_flickr.py</code>	Training (Synthetik bzw. Flickr8k, resumierbar)
<code>baseline_flickr_onnx.py</code>	Baseline: vortrainiertes CLIP via ONNX-Runtime
<code>ablation_flickr.py</code>	Experiment zum Einfluss der Batchgröße
<code>viz.py</code> , <code>viz_flickr.py</code>	Erzeugung aller Ergebnis-Abbildungen
<code>Mini_CLIP_Exploration.ipynb</code>	exploratives Logbuch (Abschn. 4.5)

Exponenzierung auf log 100 begrenzt). Der Loss entspricht Zeile für Zeile dem Pseudocode aus Figure 3 des Papers:

```
def clip_loss(logits_per_image, logits_per_text):
    n = logits_per_image.size(0)
    targets = torch.arange(n)          # korrekte Paare: Diagonale
    loss_i = F.cross_entropy(logits_per_image, targets) # Bild -> Text
    loss_t = F.cross_entropy(logits_per_text, targets)  # Text -> Bild
    return (loss_i + loss_t) / 2
```

Die zugehörigen Logits erzeugt der Forward-Pass: L2-normalisierte Embeddings, deren paarweise Skalarprodukte mit der exponenzierten, nach oben begrenzten Temperatur skaliert werden:

```
def forward(self, images, tokens):
    img = F.normalize(self.image_encoder(images), dim=-1)
    txt = F.normalize(self.text_encoder(tokens), dim=-1)
    scale = self.logit_scale.clamp(max=self.max_logit_scale).exp()
    logits_per_image = scale * img @ txt.t() # [B, B]
    return logits_per_image, logits_per_image.t()
```

Ein praktischer Nebeneffekt dieser Loss-Formulierung: Vor dem Lernen muss der Loss bei $\ln(N)$ liegen, dem Wert reinen Ratens. Genau das haben wir auch gemessen (4,67 bzw. 5,58 bei $\ln 128 \approx 4,85$ bzw. $\ln 256 \approx 5,55$). Warum der Loss kontrastiv und symmetrisch sein muss, zeigen wir im Notebook (Abschn. 2) an einem Minimalbeispiel mit 16 Konzepten; die Wirkung der Temperatur auf die Softmax-Schärfe und die Notwendigkeit der L2-Normalisierung spielen wir dort ebenfalls einzeln durch (Abschn. 5–6).

Eigene Abweichung: Text-Pooling. Das Original nutzt die Aktivierung des [EOS]-Tokens als Textrepräsentation. Wir mitteln stattdessen über alle Nicht-Padding-Tokens. Diese Entscheidung haben wir experimentell überprüft (Notebook, Abschn. 6c): Unter identischen Bedingungen erreicht die Mean-Pooling-Variante auf dem synthetischen Datensatz zuverlässig 100 % Zero-Shot-Accuracy (Epoche 7, mit zwei Seeds reproduziert), während eine nachgebaute [EOS]-Variante nach zehn Epochen erst bei rund 95 % liegt – in den ersten Epochen ist sie teils sogar vorn, konvergiert dann aber langsamer. Eine plausible Erklärung: Beim Mean-Pooling bekommt jedes Token direkt Gradientensignal, beim [EOS]-Pooling muss es durch die Attention zum letzten Token fließen – bei nur zwei Transformer-Layern ein Nadelöhr:

```
# Mean-Pooling ueber Nicht-Padding-Tokens (statt [EOS] wie im Original)
mask = (~pad_mask).unsqueeze(-1).float()
h = (h * mask).sum(1) / mask.sum(1).clamp(min=1)
return self.proj(h) # [B, embed_dim]
```

Tabelle 3: Zentrale Hyperparameter (config.py) und ihre Herkunft.

Parameter	Wert	Herkunft	Beleg
Embedding-Dimension	256	CPU-Budget	Kap. 3
Bildgröße	64px	CPU-Budget	Kap. 3
Temperatur-Init / Deckel	1/0,07 / 100	Paper	Notebook, Abschn. 5
Lernrate	$5 \cdot 10^{-4}$	Mini-Sweep	Notebook, Abschn. 6b
Weight Decay	0,2	Paper	[1]
Warmup / Schedule	50 Schritte / Cosine	Paper	Notebook, Abschn. 6b
Batchgröße	128 / 256	eigenes Experiment	Notebook, Abschn. 9
Epochen (Flickr8k)	30	Overfitting-Kurve	Notebook, Abschn. 8
Seed	42	Reproduzierbarkeit	config.py

4.3 Datenpipeline und Trainingsinfrastruktur

Für Flickr8k haben wir eine eigene Cache-Pipeline gebaut (`prep_parquet.py`): Die Parquet-Shards des Datensatzes werden einmalig dekodiert, auf 64px verkleinert und als npy/json-Cache gespeichert. Danach ist jede Trainingsepoche I/O-frei. Der Weg dorthin war mühsamer als gedacht – abbrechende Downloads, langsame Standard-Loader; was wir alles verworfen haben, steht im Notebook (Abschn. 7). Gesplittet wird auf Bildebene gegen Data Leakage; im Training nutzen wir alle fünf Captions pro Bild als sprachliche Augmentierung, evaluiert wird mit der ersten Caption.

Das Training (`train_flickr.py`) haben wir resumierbar gebaut: Nach jeder Epoche wird ein vollständiger Checkpoint gespeichert (Modell, Optimizer-Zustand, globaler Step, Tokenizer-Vokabular), und eine konfigurierbare Zeitbegrenzung bricht kontrolliert ab, statt Fortschritt zu verlieren. So konnten wir den 30-Epochen-Lauf in Etappen fahren. Den Tokenizer laden wir beim Fortsetzen bewusst aus dem Checkpoint, weil ein Neuaufbau die Vokabular-Indizierung verschieben und damit die Embedding-Tabelle unbrauchbar machen würde. Die Lernrate folgt dem selbst implementierten Schedule aus dem Paper-Rezept, also linearem Warmup und anschließendem Cosine-Decay (Visualisierung: Notebook, Abschn. 6b):

```
def lr_at(step, base, warmup, total):
    if step < warmup:
        return base * step / max(1, warmup)
    p = (step - warmup) / max(1, total - warmup)
    return base * 0.5 * (1 + math.cos(math.pi * min(1.0, p)))
```

4.4 Evaluationsdesign

Als Metriken nutzen wir Zero-Shot-Accuracy für Stufe 1 und Retrieval Recall@K für Stufe 2. Auf diese Zuordnung sind wir allerdings erst gekommen, nachdem wir selbst in eine Falle gelaufen waren: Das Recall blieb auf den synthetischen Daten niedrig, obwohl das Modell perfekt klassifizierte. Der Grund ist die Datenstruktur – bei nur 20 verschiedenen Captions kann die strikte 1:1-Metrik das „richtige“ Bild nicht von einem anderen mit identischer Caption unterscheiden. Im Notebook (Abschn. 4) haben wir das nachgestellt: Dieselben Modellvorhersagen ergeben, je nach Bewertung, ein striktes R@1 von $\approx 7\%$ oder ein caption-bewusstes R@1 von $\approx 94\%$.

4.5 Nachvollziehbarkeit: das begleitende Notebook

Das Jupyter-Notebook `Mini_CLIP_Exploration.ipynb` ist explorativ aufgebaut. Es enthält den vollständigen Modell-, Daten- und Evaluationscode inline, dokumentiert unseren Erarbeitungsweg und hinterlegt die zentralen Design-Entscheidungen mit Experimenten (Tab. 4). Ein optionaler Abschnitt am Ende (*harder Durchlauf*) reproduziert alle protokollierten Ergebnisse über die Projektskripte.

5 Experimente und Ergebnisse

5.1 Stufe 1: Korrektheitsnachweis mithilfe von „Synthetic Shapes“

Der Loss fällt vom Zufallsniveau aus monoton, und die Zero-Shot-Accuracy erreicht 100 % (Tab. 5). Das Modell ordnet also komplett ungesehene Testbilder nur über die Text-Prompts der vier Formklassen richtig zu; das Training dauerte insgesamt etwa 19s auf einer Macbook-CPU (M3). Damit ist die Implementierung nachweislich funktionsfähig. Auch die Bild-Text-Ähnlichkeitsmatrix (Abb. 4) zeigt das erwartete Bild: eine deutlich hellere Diagonale (Kosinus 0,85–0,91) als der Rest.

Tabelle 4: Behauptung in dieser Dokumentation → Beleg im Notebook.

Behauptung / Entscheidung	Experiment	Notebook
Loss braucht Negative (InfoNCE)	Kollaps-Demo, Margin-Vergleich	Abschn. 2
Metrik muss zum Datensatz passen	strikt vs. caption-bewusst	Abschn. 4
Prompt-Wortlaut wirkt (Paper 3.1.4)	Template-Vergleich & Ensembling	Abschn. 4b
Temperatur: Wirkung & Init	Softmax-Schärfe; Init 1 vs. 1/0,07	Abschn. 5
L2-Normalisierung ist Pflicht	Wertebereiche roh vs. normiert	Abschn. 6
Lernrate $5 \cdot 10^{-4}$	Sweep über 4 Größenordnungen	Abschn. 6b
Warmup hilft am Trainingsstart	konstante Lernrate vs. Warmup	Abschn. 6b
Mean- statt [EOS]-Pooling	Pooling-Vergleich, gleiche Bedingungen	Abschn. 6c
Datenbeschaffung / Cache	dokumentierte Sackgassen	Abschn. 7
Overfitting statt Bug	Loss- vs. Recall-Kurve	Abschn. 8
Phrasing vs. Informationsgehalt	Query-Varianten auf Flickr8k	Abschn. 8b
Einfluss der Batchgröße (Confound)	64/128/256 bei fester Epochenzahl	Abschn. 9
Baseline-Wahl (ONNX)	drei dokumentierte Anläufe	Abschn. 10

Tabelle 5: Trainingsverlauf auf SyntheticShapes (CPU, 1 500 Paare).

Epoche	mittl. Loss	Zero-Shot-Accuracy
1	4,67	22,0 %
3	2,38	58,3 %
6	1,96	100,0 %
8	1,95	100,0 %

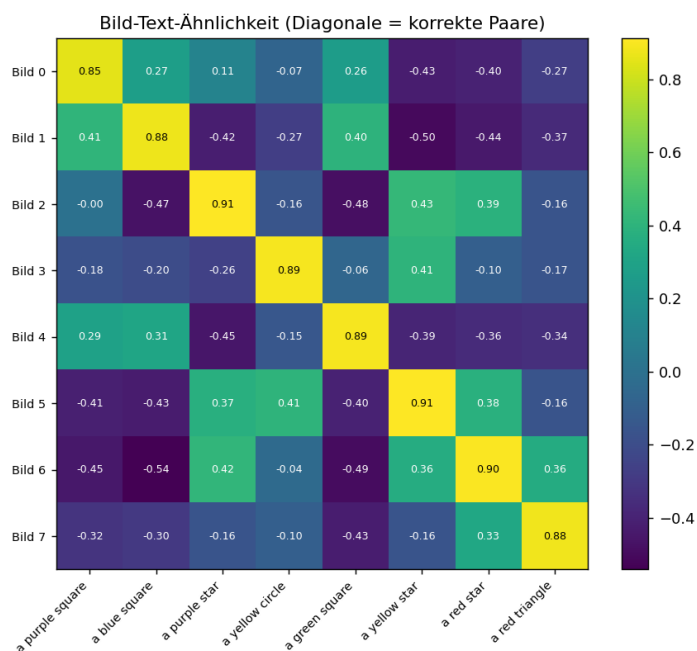


Abbildung 4: Bild-Text-Ähnlichkeitsmatrix des trainierten Mini-CLIP für acht Testpaare: Die helle Diagonale entspricht den korrekten Paaren.

Einfluss des Prompt-Wortlauts. In Anlehnung an Abschn. 3.1.4 des Papers wollten wir außerdem wissen, wie stark die Formulierung der Prompts das Ergebnis beeinflusst (Notebook, Abschn. 4b). Weil unser voll trainiertes Modell die 4-Klassen-Aufgabe schon mit 100 % löst, sind wir auf die schwierigere 20-Klassen-Aufgabe (Farbe $\times$ Form) und einen untertrainierten Zwischenstand (3 Epochen) ausgewichen und haben über drei Seeds gemittelt. Das Template im Trainings-Format kam im Mittel auf 31,4 %, der nackte Klassenname auf 25,9 % – dieselbe Richtung wie die +1,3 Punkte im Paper. Templates mit Wörtern, die nicht im Vokabular stehen („picture“ wird zu `<unk>`), fielen auf 25,8 %. Durch das geschlossene Vokabular reagiert unser Mini-CLIP also empfindlicher auf den Wortlaut als das Original. Prompt-Ensembling brachte bei uns mit 29,1 % keinen Vorteil gegenüber dem besten Einzel-Template. Ein Gegenexperiment auf Flickr8k (Notebook, Abschn. 8b) ordnet das ein: Dort sind die Queries schon vollständige Sätze aus der Trainingsverteilung, und Templates ändern am Retrieval praktisch nichts ($\sim 5\text{--}6\%$ R@10 über alle Varianten). Kürzt man die Query dagegen auf fünf Wörter, fällt der Recall von 5,3 % auf 3,1 %. Der Prompt-Trick des Papers hilft also vor allem dann, wenn die Query unnatürlich kurz ist – das Template macht aus einem Klassennamen einen satzartigen Text, wie ihn der Encoder aus dem Training kennt.

5.2 Stufe 2: Flickr8k und Overfitting

Auf Flickr8k (30 Epochen, 6 000 Bilder, Batchgröße 256) fällt der Train-Loss kontinuierlich von 5,58 auf 1,24, während das Test-Retrieval ab etwa Epoche 5 bei ca. 6 % Recall@10 stagniert (Abb. 5). Das lässt auf Overfitting schließen: Das Modell merkt sich die Trainingspaare, generalisiert aber kaum. Es liegt zwar deutlich über dem Zufall (R@10: 6,0 % vs. 1,0 %), aber weit von brauchbarer Qualität entfernt. Ein Implementierungsfehler kann zwar nicht ausgeschlossen werden, ist jedoch unwahrscheinlich, weil Stufe 1 die Korrektheit belegt. Für uns ist das Ergebnis vielmehr die erwartbare Folge von nur 6 000 Trainingsbildern – und damit schon der erste empirische Beleg für die Skalierungsaussage des Papers.

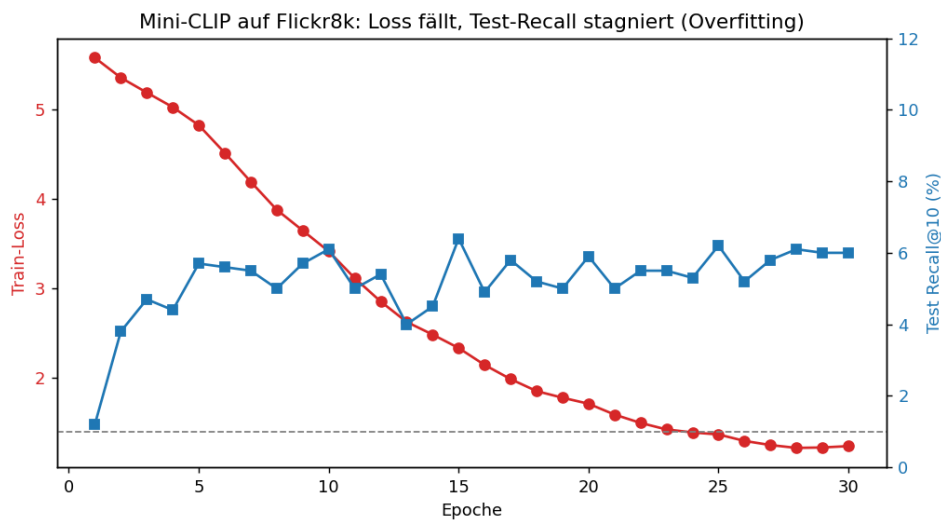


Abbildung 5: Trainingsverlauf auf Flickr8k: Der Train-Loss fällt kontinuierlich, der Test-Recall stagniert ab ca. Epoche 5 – klassisches Overfitting.

5.3 Baseline-Vergleich und Einfluss der Batchgröße

Um unsere Zahlen einordnen zu können, haben wir das vortrainierte CLIP ViT-B/32 auf exakt demselben Test-Split ausgewertet. Der Weg dorthin war holprig: `open_clip` scheiterte an einer `torchvision`-Inkompatibilität unserer Umgebung, deshalb sind wir pragmatisch auf quantisierte

Tabelle 6: Bild→Text-Retrieval auf identischem Flickr8k-Test-Split (1 000 Bilder).

Modell	Trainingsdaten	R@1	R@5	R@10
Zufall	—	0,1 %	0,5 %	1,0 %
Mini-CLIP (from scratch)	6 000 Paare	0,9 %	2,9 %	6,0 %
CLIP ViT-B/32 (pretrained)	~400 Mio. Paare	40,0 %	66,5 %	76,7 %

ONNX-Encoder mit ONNX-Runtime ausgewichen. Da Prinzip und Loss identisch sind, isoliert der Vergleich den Skalierungseffekt: R@1 40 % vs. 0,9 % bei ca. 67 000-facher Datenmenge und ca. 110-facher Modellgröße (Tab. 6, Abb. 6). Zusätzlich haben wir getestet, wie sich die Batchgröße

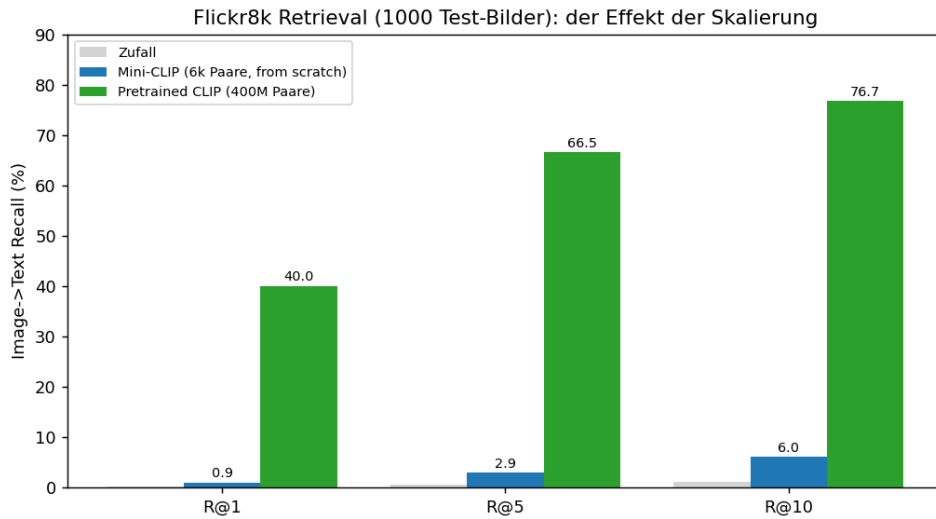


Abbildung 6: Mini-CLIP gegen vortrainiertes CLIP ViT-B/32 auf identischem Test-Split: identische Mechanik, großer Abstand durch Skalierung.

in unserem kleinen Maßstab auswirkt (identische Daten, Epochen und Seed; nur die Batchgröße variiert: 64/128/256). Das Ergebnis hat uns zunächst überrascht: Der größte Batch schnitt am schlechtesten ab (R@10: 5,1 / 5,6 / 4,1 %). Bei der Analyse sind wir auf einen sogenannten Confound gestoßen – wir hatten ungewollt zwei Dinge gleichzeitig verändert: Bei fester Epochenzahl macht ein größerer Batch nämlich auch weniger Gewichts-Updates (312 → 72 Schritte). Ob das schlechte Ergebnis an der Batchgröße selbst liegt oder an den fehlenden Updates, lässt sich in diesem Aufbau nicht trennen; bei so wenig Daten wiegen die fehlenden Updates vermutlich schwerer als der Vorteil zusätzlicher Negative. Der im Paper beschriebene Nutzen großer Batches setzt also Skalierung voraus. Eine saubere Wiederholung mit konstant gehaltener Schrittzahl haben wir uns als nächsten Schritt notiert.

6 Diskussion und Fazit

Unser Projekt reproduziert die Mechanik von CLIP qualitativ vollständig und belegt die Kernaussage des Papers im kontrollierten Kleinexperiment: Dieselbe Mechanik liefert mit 6 000 Paaren ca. 6 % R@10, mit 400 Millionen Paaren 76,7 %. Die Fähigkeiten des Originals kommen also aus der Skalierung und nicht aus der (bewusst einfachen) Architektur. Zwei Limitationen des Papers zeigen sich auch bei uns: die Abhängigkeit vom Prompt-Wortlaut und die schwache Generalisierung außerhalb der Trainingsverteilung. Unsere Untersuchung hat natürlich selbst Grenzen: Es sind größtenteils Einzelläufe mit festem Seed (nur das Prompt-Experiment haben wir über drei Seeds gemittelt), der Wort-Tokenizer kennt nur die Wörter aus den Trainings-Captions, und

beim Batchgrößen-Experiment haben wir – wie in Abschn. 5.3 beschrieben – mit der Batchgröße ungewollt auch die Zahl der Gewichts-Updates mitverändert, sodass sich die beiden Effekte nicht sauber trennen lassen.

Methodisch haben uns zwei einfache Kontrollen am meisten geholfen. Die erste ist der synthetische Datensatz: Weil dort die richtige Lösung von vornherein bekannt ist, konnten wir bei jedem schlechten Ergebnis sofort unterscheiden, ob unser Code fehlerhaft ist oder ob einfach die Datenmenge nicht reicht (Abschn. 5.1). Die zweite ist der Blick auf den allerersten Loss-Wert: Vor dem Training muss er beim Zufallswert $\ln(N)$ liegen – weicht er davon ab, stimmt etwas Grundlegendes nicht (Abschn. 4.2). Dazu kommt eine allgemeine Erfahrung: Metriken und Hyperparameter-Empfehlungen gelten immer nur für den Maßstab, in dem sie entstanden sind, und lassen sich nicht blind übertragen. Den vollständigen Erarbeitungsweg mit allen Zwischenexperimenten dokumentiert das begleitende Jupyter-Notebook (Zuordnung in Tab. 4).

7 Einsatz von KI-Werkzeugen

Bei diesem Projekt wurde an mehreren Stellen Künstliche Intelligenz eingesetzt: um einzelne Komponenten zu implementieren, Code zum Laufen zu bringen und zu debuggen und in Sackgassen Lösungsansätze zu bekommen, außerdem für Formatierungsarbeiten (Texte, $\text{\LaTeX}$ -Code, Rechtschreibung) und für die Erstellung einer Grafik. Neben Claude (Anthropic) und ChatGPT (OpenAI) kamen dabei auch GitHub Copilot (Autovervollständigung und Autofix-Vorschläge direkt im Editor) sowie die in Overleaf integrierte Schreibassistent (Writefull) zum Einsatz, mit der Texte Korrektur gelesen und grammatikalisch überarbeitet wurden. Die folgenden Auflistungen schildern den Einsatz möglichst transparent.

KI-generiert ist ausschließlich Abbildung 3 (das Architekturdiagramm), erstellt nach eigener Spezifikation. Alle übrigen Abbildungen stammen entweder direkt aus dem Paper [1] (Abb. 1) oder wurden mit den eigenen Skripten (`viz.py`, `viz_flickr.py`; beim Schreiben half die Copilot-Autovervollständigung) aus unseren Messdaten erzeugt (Abb. 2, 4, 5 und 6).

Tabelle 7: Überblick über den KI-Einsatz nach Projektbereich.

Bereich	Art der Nutzung
Konzeption & Design (Kap. 3)	keine
Implementierung (<code>src/</code> , Skripte)	Hilfe beim Lauffähig-Machen (Umgebungsprobleme)
Fehlersuche in Sackgassen	Ursachenanalyse und Lösungsvorschläge auf konkreten Fehlern
Programmierung (Editor)	Autovervollständigung und Autofix mit GitHub Copilot
Tokenizer (<code>src/tokenizer.py</code>)	vollständig von Claude implementiert, anschließend
Daten-Hilfsklassen (<code>prep_parquet.py</code> , <code>CachedFlickr</code> , <code>Collate</code>)	mit ChatGPT-Unterstützung nach unseren Anforderungen
Ergebnisabbildungen (<code>viz.py</code> , <code>viz_flickr.py</code>)	eigene Skripte, Copilot-Autovervollständigung
Experimente & Auswertung (Kap. 5)	keine
Dokumentation (dieses Dokument)	Strukturierung und Ausformulierung, danach überarbeit
Notebook (Abschn. 4.5)	Umsetzung der Demo-Experimente nach Vorgabe, T
Korrekturlesen (gesamtes Dokument)	Grammatik/Rechtschreibung mit Overleaf-KI (Writefull)
Präsentation	Foliengestaltung und Layout, Inhalte vorgegeben
Abb. 3	nach eigener Spezifikation KI-generiert

Im Folgenden werden die Prompts sinngemäß oder wörtlich wiedergegeben, die bei der Erstellung dieses Dokuments verwendet wurden (Copilot – Autovervollständigung im Editor – und die Overleaf-Schreibassistent wurden ebenfalls verwendet, arbeiten aber ohne Prompts und erscheinen daher nur in der Tabelle):

1. **Abschn. 5.3, Baseline** (`baseline_flickr_onnx.py`) (Claude)
Zweck: Debugging der Baseline, nachdem `open_clip` an einer `torchvision`-Inkompatibilität scheiterte

- Prompt (sinngemäß):* „Meine pretrained-CLIP-Baseline läuft wegen einer torchvision-Inkompatibilität nicht auf meinem CPU-Setup – wie bekomme ich die beiden Encoder alternativ (z. B. als ONNX) zum Laufen? Open Clip funktioniert leider nicht (... Fehlermeldung...). Gibt es Alternativen, und wie bekomme ich es im Stil dieses Codes zum Laufen? (... bisheriger Code...)“
2. **gesamtes Dokument** (Claude)
Zweck: Strukturierung und Ausformulierung der Dokumentation auf Basis von Repo, Messwerten und inhaltlichen Vorgaben
Prompt (sinngemäß): „Kannst du dir die aktuelle Struktur anschauen, ist diese verständlich genug?; Fehlen Inhalte, die im Paper behandelt wurden?; Habe ich CLIP vollständig erklärt?; Korrigiere Rechtschreibfehler / L^AT_EX-Fehler / Formatierungen / Satzstruktur“
 3. **Abb. 3** (Claude)
Zweck: Erstellung des Architekturdiagramms nach eigener Skizze
Prompt (sinngemäß): „Erstelle ein Architekturdiagramm meines Mini-CLIP mit zwei Encoder-Pfaden, linearer Projektion, $N \times N$ -Ähnlichkeitsmatrix mit markierter Diagonale.“
 4. **src/tokenizer.py** (Claude)
Zweck: vollständige Implementierung des Wort-Tokenizers nach unserer Spezifikation
Prompt (sinngemäß): „Implementiere einen einfachen Wort-Tokenizer für unser Mini-CLIP: Kleinschreibung, alphanumerische Tokens, Vokabular aus den Trainings-Captions mit Mindesthäufigkeit, Tokens wie <pad>/<eos> und den Eigenschaften, die du für eine Mini-CLIP-Umsetzung sinnvoll hältst. => Weitere Verbesserungen des Tokenizers durch Vibe Coding“
 5. **Daten-Hilfsklassen (prep_parquet.py; CachedFlickr/Collate in src/data.py)** (ChatGPT)
Zweck: Unterstützung beim Erstellen der Hilfsklassen für Datenaufbereitung und Laden – unsere Anforderungen kamen aus den Problemen der Datenbeschaffung (Notebook, Abschn. 7)
Prompt (sinngemäß): „Der Flickr8k-Download bricht ständig ab und das Training ist langsam, weil jede Epoche die JPEGs neu dekodiert. Wir wollen: die Parquet-Dateien einmalig einlesen, alle Bilder auf 64 px verkleinern und als npy/json-Cache speichern; ein Dataset, das beim Training zufällig eine der fünf Captions zieht, bei der Evaluation aber immer die erste; Split auf Bildebene, damit nichts leakt. Schreib uns dafür passende Klassen. => Code übernommen und angepasst“
 6. **Training (train_flickr.py; Notebook, Abschn. 6b)** (Claude)
Zweck: Frage, warum das Training am Anfang instabil lief und wie das Paper das löst; => daraus übernehmen wir Warmup + Cosine-Decay
Prompt (sinngemäß): „Unser Loss springt in den ersten Trainingsschritten stark und wird zeitweise schlechter als Raten (die KI hat hier Bilder und Konsolenausgaben der Auswertung bekommen) – woran kann das liegen, und wie haben die CLIP-Autoren das gelöst?“
 7. **diverse Sackgassen (Notebook, Abschn. 2, 4, 7 und 10)** (Claude)
Zweck: Lösungsansätze, wenn wir feststeckten – z. B. kollabierender Loss, irreführende Metrik, abbrechende Downloads der Flickr-Daten
Prompt (sinngemäß): „Wir stecken an dieser Stelle fest (Code/Fehlerbild anbei) – welche Ursachen kommen infrage und wie kann man das umgehen?“
 8. **Notebook, Abschn. 4 (Metrik-Demo)** (Claude)
Zweck: Umsetzung einer Demo-Zelle zur irreführenden Retrieval-Metrik auf den synthetischen Daten
Prompt (sinngemäß): „Das Recall-Ergebnis war schlechter als erwartet. Prompt an die KI:

Ist das Ergebnis tatsächlich so schlecht oder kann man es verbessern? => Die KI antwortete, es könne am Handling der Captions liegen, und hat den Code entsprechend korrigiert. Anschließend haben wir sie gebeten, Code zu schreiben, der den Unterschied demonstriert, und diesen im Notebook verwendet.“

9. **Notebook, Abschn. 5 und 6b (Hyperparameter)** (Claude)

Zweck: kleine Experimente, die unsere Wahl von Lernrate und Temperatur-Startwert belegen

Prompt (sinngemäß): „Kannst du einen Lernraten-Sweep über mehrere Größenordnungen und eine Gegenprobe zum Temperatur-Startwert aus dem Paper umsetzen? (Hieraus wurde Code verwendet)“

10. **Notebook, Abschn. 4b und 8b (Prompt-Experimente)** (Claude)

Zweck: Umsetzung der Prompt-Vergleiche auf Synthetik und Flickr8k nach unserer Idee

Prompt (sinngemäß): „Im Paper verbessert besseres Phrasing die Genauigkeit – das will ich auch probieren. Teste verschiedene Prompt-Templates, und prüfe das anschließend auch auf den Flickr-Daten. (Hiervon wurde auch Code verwendet)“

11. **Notebook, Text und Abschn. 3/12** (Claude)

Zweck: Code aus `src/` in die Zellen übernehmen, Markdown-Erklärungen überarbeiten

Prompt (sinngemäß): „Überprüfe das Notebook auf Vollständigkeit gegenüber `src/`. Gab es Experimente, die wir vergessen haben oder einfügen sollten?; Liste auf, welche Zellen unvollständig dokumentiert sind; Überarbeite Rechtschreibung / Satzformulierung; Prüfe die Struktur des Notebooks => daraus entstand der harte Durchlauf am Ende“

Literatur

- [1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G. & Sutskever, I. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. arXiv:2103.00020. <https://arxiv.org/abs/2103.00020>
- [2] Oord, A. van den, Li, Y. & Vinyals, O. (2018). *Representation Learning with Contrastive Predictive Coding*. arXiv:1807.03748.
- [3] Dosovitskiy, A. et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929.
- [4] Hodosh, M., Young, P. & Hockenmaier, J. (2013). *Framing Image Description as a Ranking Task: Data, Models and Evaluation Metrics*. Journal of Artificial Intelligence Research 47, 853–899.